

BCSE307P- Compiler Design Lab

Project Review 1 – Report Submission

Project Title: CodeGuardian: A Compiler-Based Static Security Analysis Engine

Team Member(s):

S. No	Register No	Name	Contributions
1.	24BCE0686	Mayur R Keni	Compiler Core: Theory & Front-End, Modules 1-5 (Flex & Bison parsing, Symbol Table, AST lowering, TAC syntax-directed rules, Leader identification, CFG generation).
2.	24BCE0571	Nitin Yajatkumar	Security Intelligence: Math Engine & Full-Stack, Modules 5-6 (Data-flow equations, iterative fixpoint calculation, Source/Sanitizer/Sink taint tracking, Backend integration, UI dashboard).

Problem Identification & Objectives – 2M

Problem Statement

High-severity zero-day injection flaws (such as SQLi and RCE) continue to threaten systems because untrusted user inputs enter execution sinks unvalidated. Current dynamic application security testing (DAST) incurs high remediation overhead by detecting flaws late in the execution phase. Meanwhile, commercial static analysis tools (SAST) operate as “black boxes,” offering line-number alerts without showing the data-flow propagation paths through loops and blocks. This project bridges the theoretical disconnect in traditional academic compilers by leveraging data structures (AST, TAC, CFG) for proactive, transparent software security.

Objectives (Minimum 3)

- Construct a custom Mini-C compiler pipeline utilizing Flex and Bison to parse imperative constructs into an Abstract Syntax Tree (AST).
- Implement Syntax-Directed Translation (SDT) to lower the AST into linearized Three-Address Code (TAC) quadruples.
- Synthesize explicit Control Flow Graphs (CFG) by partitioning TAC sequences into basic blocks using canonical leader analysis.
- Formulate a monotone data-flow taint engine that evaluates iterative fixpoint transfer functions across basic blocks to track source-to-sink variable propagation.
- Deliver a real-time, explanatory security dashboard that renders the generated CFG and visually highlights vulnerable paths.

Literature Survey – 2M

S.No	Title / Author / Year	Methodology	Findings	Gap Identified
1	Rizk, Ben Hmida & Eshete, "TaintRadar: Semantic-Aware Taint-Style Vulnerability Detection via Augmented Code Property Graphs," 2024	Augments Code Property Graphs with a compositional semantic layer modeling sanitization, database persistence and object aliasing.	Deployed on 19 real-world PHP applications and uncovered 29 confirmed zero-day vulnerabilities (26 SQLi, 3 stored XSS).	Still a graph-level black box to the developer; the augmented CPG is not surfaced as a teachable, inspectable compiler intermediate form.
2	Yuan, Lu, Zhu, Huang, Yu & Zhao, "A Static Detection Method for SQL Injection Vulnerability Based on Program Transformation," Applied Sciences, 2023 (tool: OODBE-SCAN)	Transforms object-oriented database extension calls into semantically equivalent procedural form, then applies CFG construction plus taint analysis.	Detected more real-world SQL-injection vulnerabilities with higher accuracy than the RIPS and Seay tools across eight PHP applications.	Narrowly scoped to object-oriented database-extension flaws rather than a general-purpose compiler-based security pipeline.
3	Semgrep (r2c / Semgrep Inc., first released 2020)	Tree-sitter AST pattern matching with user-defined syntactic rules.	Expressive, language-agnostic rule syntax with fast incremental scanning suited to modern CI pipelines.	Shallow, largely intraprocedural data-flow tracking; loses precision across complex loops and merging conditional branches.
4	GitHub CodeQL (GitHub, acquired/relaunched 2019)	Translates source code into a relational database queried with the Datalog-style QL language.	Enterprise-grade interprocedural analysis capable of expressing complex taint and security queries.	Steep learning curve, heavy compute requirements, and a closed, opaque intermediate representation not exposed to the end user.
5	Ernst, Lovato, Macedonio, Spiridon & Spoto, "Boolean Formulas for the Static Identification of Injection Attacks in Java," LPAR, 2015 (tool: Julia)	Flow-, context-, object- and field-sensitive analysis of Java bytecode, encoded as Boolean formulas representing all possible explicit taint flows.	Sound analysis that discovered real injection vulnerabilities in production banking and CRM software.	Operates on compiled bytecode with heavyweight Boolean-formula solving; offers no human-readable intermediate structure for learning or verification.
6	Arzt, Rasthofer, Fritz, Bodden et al., "FlowDroid: Precise Context, Flow, Field, Object-sensitive and Lifecycle-aware Taint Analysis for Android Apps," PLDI, 2014	Context-, flow-, field- and object-sensitive static taint analysis built on a precise model of the Android component lifecycle.	Achieved 93% recall and 86% precision on the DroidBench benchmark, outperforming commercial tools such as Fortify SCA and IBM AppScan Source.	Domain-specific to the Android application lifecycle and callback model; not directly applicable to generic imperative source languages.
7	Jovanovic, Kruegel & Kirda, "Static analysis for detecting taint-style vulnerabilities in web applications," Journal of Computer Security, 2010 (tool: Pixy)	Flow-sensitive taint analysis with a precise alias model for PHP reference semantics and an iterative two-phase file-inclusion resolver.	High-precision detection of SQL injection, XSS and command injection in real PHP applications with improved report quality.	Tied to PHP scripting semantics and reference handling; does not expose an explicit, inspectable CFG/TAC-level intermediate representation.
8	OWASP ZAP (Dynamic Application Security Testing, first released 2010)	Runtime fuzzing and interception proxy that actively attacks a running application.	Confirms genuinely exploitable vulnerabilities under real execution conditions with low false positives.	Detects flaws only late, during or after deployment, so remediation cost is high and code-path coverage is incomplete.

S.No	Title / Author / Year	Methodology	Findings	Gap Identified
9	SonarQube (SonarSource, first released 2008)	Rule-based static code-quality and security-hotspot scanning with limited cross-procedure data-flow rules.	Wide language support and strong CI/CD/dashboard integration for continuous code-quality tracking.	Primarily pattern/rule based; interprocedural taint propagation and explicit control-flow visualization are minimal.
10	Wassermann & Su, “Sound and Precise Analysis of Web Applications for Injection Vulnerabilities,” PLDI, 2007	Models string variables with context-free grammars and string operations as language transducers to detect syntactic-structure violations from user input.	Fully automated, sound SQL-injection detector with a low false-positive rate that scales to ~100K LOC PHP programs.	Requires hand-built transducer models for each sanitizer/string operation and is not generalizable to an arbitrary compiler IR.
11	Xie & Aiken, “Static Detection of Security Vulnerabilities in Scripting Languages,” USENIX Security, 2006	Precise, block-level taint tracking that checks only for dangerous content at the specific point of use rather than tainting an entire derived value.	Reduces false positives compared to coarse whole-value tainting by tracking taintedness within data values.	Analysis remains at the source-code text level; propagation paths are not visualized or mapped onto an intermediate representation.
12	Commercial SAST suites: Fortify SCA, Checkmarx, Coverity (mid-2000s onward)	Proprietary interprocedural dataflow and pattern-matching engines integrated into build systems and CI/CD pipelines.	Broad language and vulnerability-class coverage with mature enterprise tooling and reporting.	Closed-source, license-restricted “black boxes” that report only line-number alerts without exposing AST/TAC/CFG-level propagation evidence.
13	Flawfinder / RATS (open-source lexical scanners, early 2000s)	Regex-based lexical pattern matching against known dangerous function signatures.	Extremely fast with minimal resource overhead; simple to deploy on any codebase.	Context-unaware; cannot distinguish sanitized from unsanitized use, causing very high false-positive rates and no tracking of data flow across basic blocks.
14	Kam & Ullman, “Monotone Data Flow Analysis Frameworks,” Acta Informatica, 1977	Generalizes Kildall’s lattice approach to monotone (not necessarily distributive) data-flow frameworks and proves convergence of the maximal-fixed-point solution.	Shows the maximal fixed point is computable by Kildall’s algorithm for any monotone framework, formally justifying iterative worklist solvers.	Remains a purely theoretical result; provides no applied security instantiation such as source/sanitizer/sink semantics.
15	Kildall, “A Unified Approach to Global Program Optimization,” ACM POPL, 1973	Lattice-theoretic iterative fixpoint algorithm driven by per-node transfer functions and a meet operator over a control-flow graph.	Established the foundational unified framework underlying constant propagation, common sub-expression elimination and live-variable analysis.	Purely a compiler-optimization framework; the original work does not apply the fixpoint machinery to security taint propagation.

Gap Identified

Current security-analysis methodologies range from fast lexical matchers that lack context, through AST-pattern and CPG-based scanners with only shallow or graph-level data-flow visibility, to enterprise-grade relational engines and commercial SAST/DAST suites that are either resource-heavy and opaque or detect flaws only late at runtime. Foundational compiler theory (Kildall; Kam & Ullman) supplies a rigorous monotone fixpoint framework, but it has never been directly repurposed, in a transparent and pedagogically inspectable form, as a source-to-sink taint engine. The primary research gap is therefore the absence of a strict, deterministic pipeline that bridges classical compiler data-flow frameworks with a transparent intermediate-representation (IR) mapping. Existing tools fail to provide interactive, visual verification of taint paths traversing intermediate compiler structures like TAC and CFG — the gap CodeGuardian is built to close.

Novelty – 2M

- **Zero-Dependency Core:** The pipeline is engineered directly from fundamental theory using custom C/C++ lowering logic and Flex/Bison, completely avoiding the use of third-party parser generator wrappers.
- **Transparent IR Taint Propagation:** Unlike commercial scanners that analyze raw source code, this engine linearizes high-level control structures into TAC quadruples, accurately tracking taint state across intermediate temporary registers.
- **Visual Data-Flow Verification:** The tool unifies graph theory with an interactive UI, enabling developers to inspect basic block leaders, TAC lowering, and the exact set convergence of vulnerable control paths in real-time.
- **Direct Theoretical Mapping:** The project directly translates formal, abstract textbook data-flow equations (Kildall's monotone fixpoint framework) into an operational, industrial-style security verification engine.

Architecture Diagram:

